White Box & Black Box Testing

Testing Principles

- **All tests should be traceable to customer requirements.**
- **Tests should be planned long before testing begins.**
- **The Pareto principle applies to software testing.**
  - ***Pareto Principle*** *is commonly interpreted as "80% of all bugs can be found in 20% of program modules"*
- **Testing should begin "in the small" and progress toward testing "in the large."**
- **Exhaustive testing is not possible.**
- **To be most effective, testing should be conducted by an independent third party.**

Testing attributes

1. A good test has a high probability of finding an error.
   - Tester must understand the software and attempt to develop a mental picture of how the software might fail.
2. A good test is not redundant.
   - Testing time and resources are limited.
   - There is no point in conducting a test that has the same purpose as another test.
   - Every test should have a different purpose
   - Ex. Valid/ invalid password.
3. A good test should be “best of breed”
   - In a group of tests that have a similar intent, time and resource limitations may mitigate toward the execution of only a subset of these tests.
4. A good test should be neither too simple nor too complex.
   - sometimes possible to combine a series of tests into one test case, the possible side effects associated with this approach may mask errors.
   - Each test should be executed separately

White box testing

Using this method, SE can derive test cases that

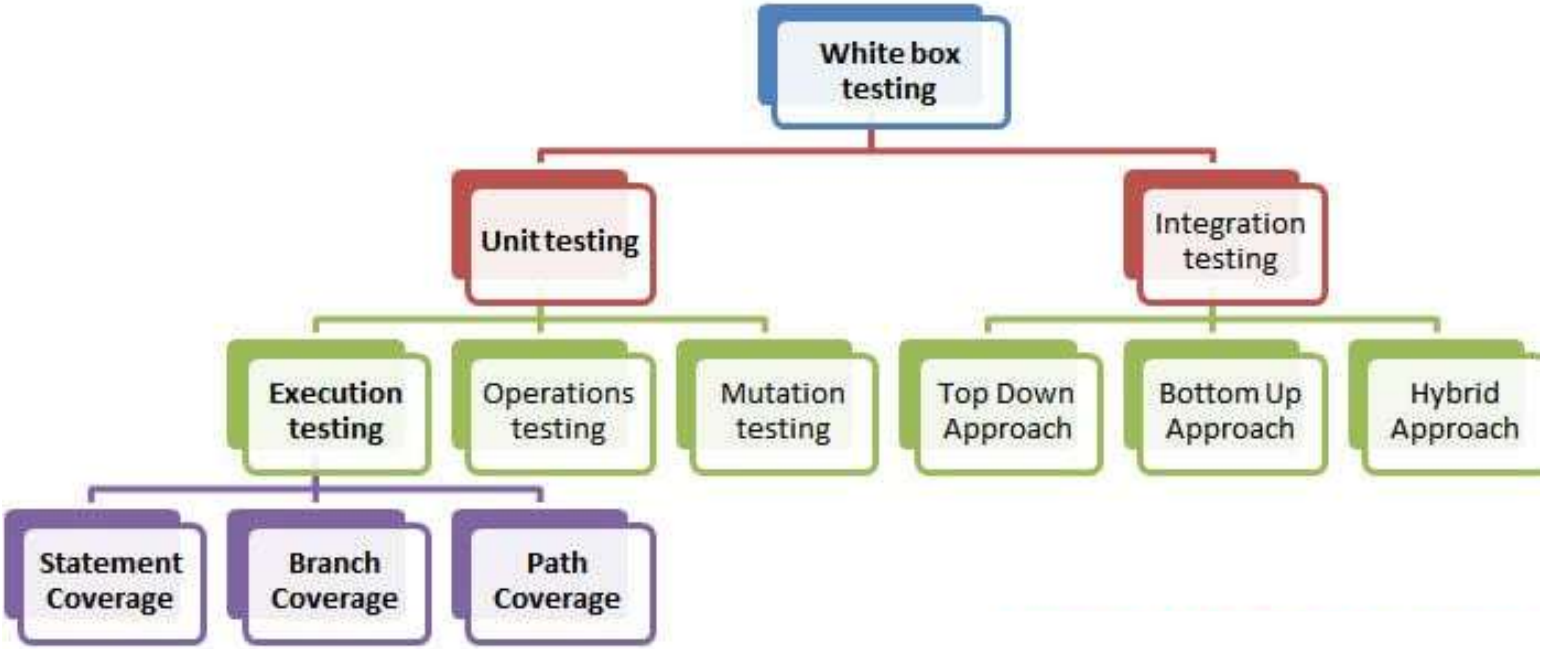
1. Guarantee that all independent paths within a module have been exercised at least once

1. Exercise all logical decisions on their true and false sides,

1. Execute all loops at their boundaries and within their operational bounds

1. Exercise internal data structures to ensure their validity.

Types of White Box Testing



White Box Testing

Coverage based Tests

It is critical to have good test coverage to ensure the quality of the product you are delivering.

- Statement Coverage
- Branch Coverage
- Condition Coverage
- Path Coverage

Fault based testing

- This strategy targets to detect certain types of faults
- These faults constitute fault model

Eg: mutation testing

Statement Coverage Testing

- Aims to design test cases so as to execute every statement in a program at least once.
- Paths executed during the testing include all nodes in the graph.

$$\text{Statement coverage} = \frac{\text{Number of executed statements}}{\text{Total number of statements}} * 100$$

- To design test cases for the statement coverage, the conditional expression of the while stmt needs to be made true

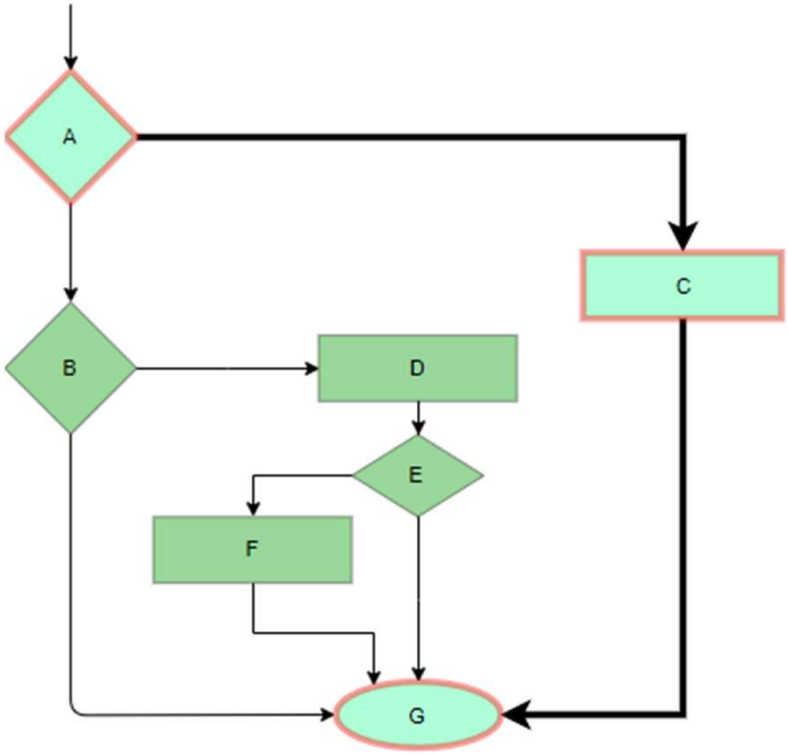
- and the conditional expression of the if stmt needs to be made both true and false

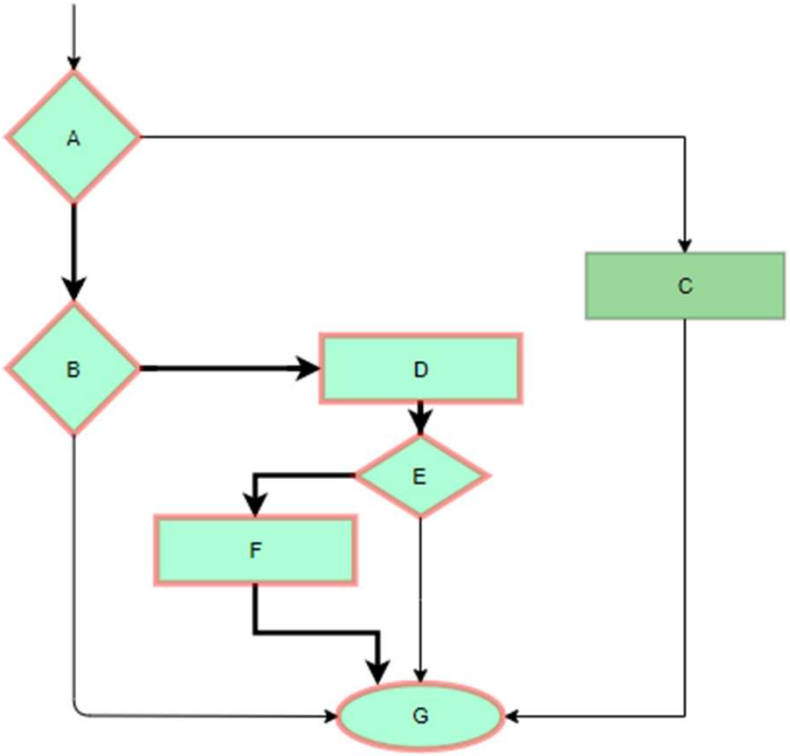
Sample test suit
{(x=3, y=3), (x=4, y=3), (x=3, y=4)},

The above test case allows the execution of every stmt at least once

- Example:
- Consider the Euclid's GCD computation algorithm:

```
int compute_gcd(x, y)
     int x, y;
{
   1    while (x! = y){
  2         if (x>y) then
   3                  x= x – y;
  4         else  y= y – x;
  5   }
  6   return x;
}
```


A
C
B
D
E
F
G


A
C
B
D
E
F
G

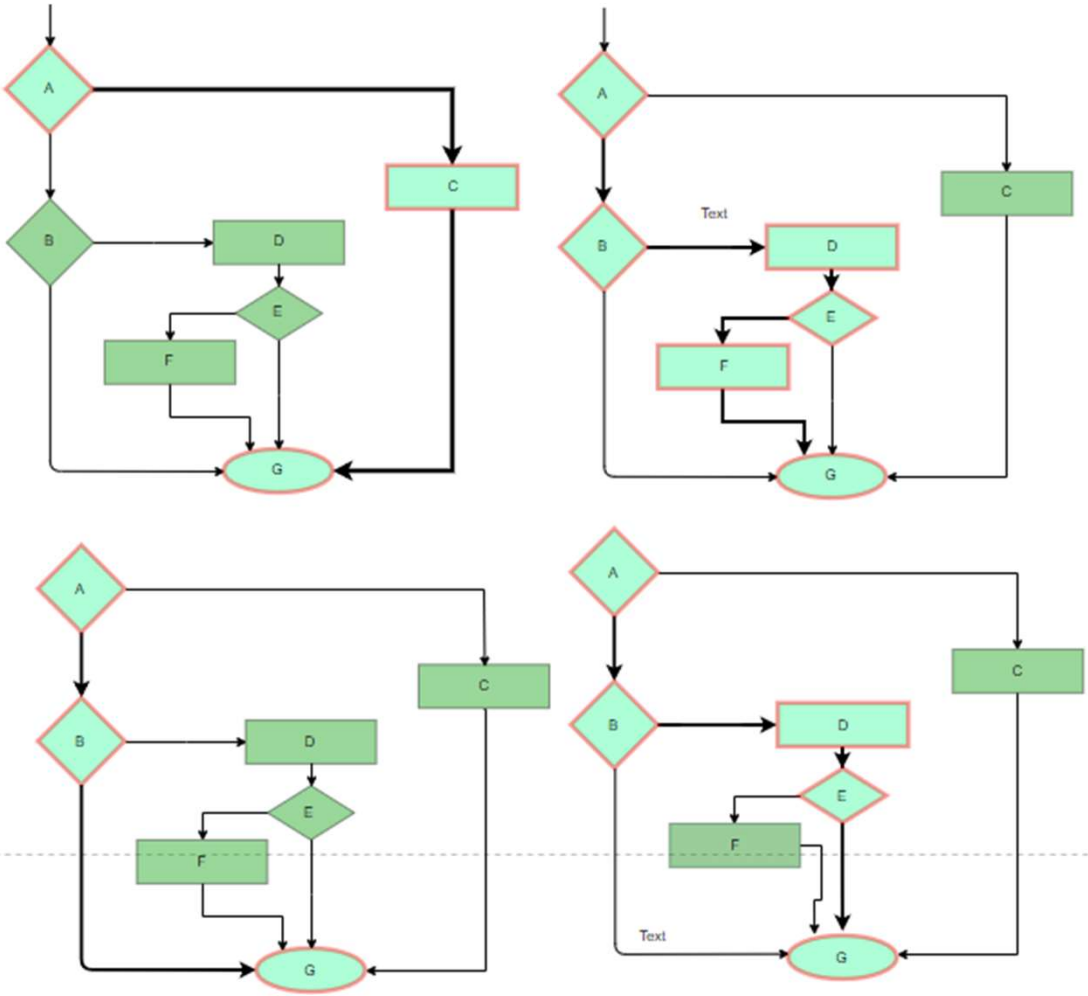
Branch Coverage (Edge testing)

- Requires test cases to be designed so as to make each branch condition in the program to assume true and false values in turn.

the test cases for branch coverage can be
{
(x=3, y=3),
(x=3, y=2),
(x=4, y=3),
(x=3, y=4)
}.

Branch testing is stronger than stmt testing

```
int compute_gcd(x, y)
    int x, y;
{
   1   while (x! = y){
  2       if (x>y) then
   3                x= x – y;
  4       else  y= y – x;
  5  }
  6  return x;
}
```


A
C
B
D
E
F
G
A
C
Text
B
D
E
F
G
A
C
B
D
E
F
G
A
C
B
D
E
F
Text
G

Condition coverage

- Condition coverage is also known as Predicate Coverage
- Condition coverage is seen for Boolean expression, ensures whether all the Boolean expressions have been evaluated to both TRUE and FALSE

```
if ( (A || B) && C )
        {
           /* instructions */
        }
        else
        {
            /* instructions */
        }
```

where A, B and C represent atomic boolean expressions (i.e. not divisible in other boolean sub-expressions).

In order to ensure Condition coverage criteria for this example, A, B and C should be evaluated at least one time "true" and one time "false" during testes.

So, in our example, the 3 following testes would be sufficient to valid Condition coverage:

A = true / B = not eval / C = false
A = false / B = true / C = true
A = false / B = false / C = not eval

Path Coverage

- Requires designing test cases such that all linearly independent paths in the program are executed at least once.

- All possible control paths taken, including
  - **All loop paths** taken **zero**, **once** and **multiple items** in technique
  - The **test case** are **prepared** based on the **logical complexity measure** of the procedure design
- **Flow graph**, **Cyclomatic Complexity** and **Graph Metrices** are used to arrive at basis path.

Basis path testing

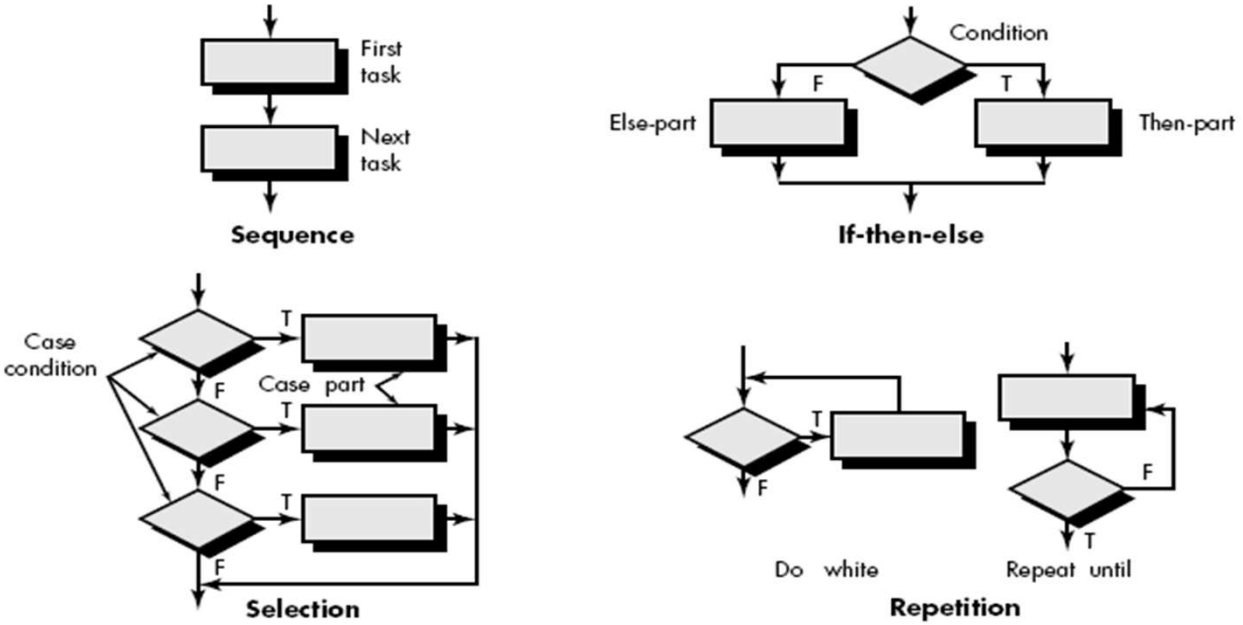
- *Basis path testing* is a white-box testing technique
- Every possible block of code in a program is executed through the lowest number of test cases.
- To derive a logical complexity measure of a procedural design.
- Test cases derived to exercise the basis set are guaranteed to execute every statement in the program at least one time.

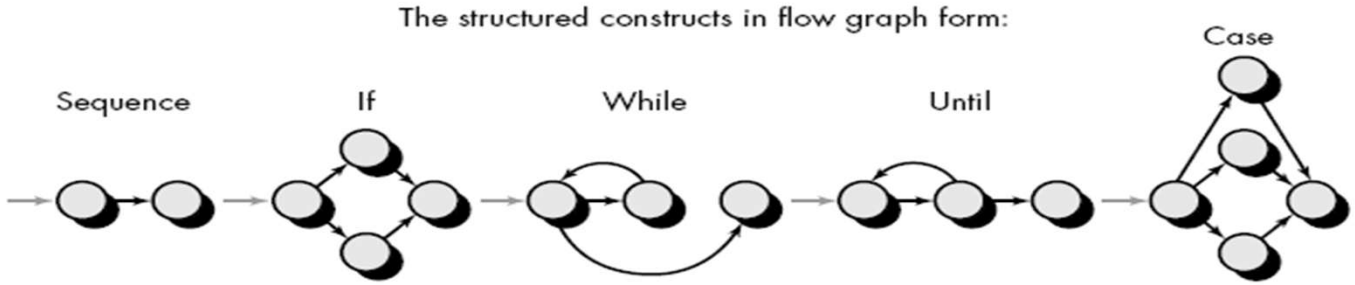
Methods:

1. Flow graph notation
2. Independent program paths or Cyclomatic complexity
3. Deriving test cases
4. Graph Matrices

1. Flow Graph Notation

- Start with simple notation for the representation of *control flow (called flow graph).* It represent logical control flow.





- When we translating PDL (Program Design Language) segment into flow graph, separate node is created for each condition.

- Each node that contains a condition is called *predicate node* and is characterized by two or more edges comes from it.

- A sequence of process boxes and decision diamond can map into a single node.
- The arrows on the flow graph, called edges or links, represent flow of control and are parallel to flowchart arrows.
- An edge must terminate at a node, even if the node does not represent any procedural statement.
- Areas bounded by edges and nodes are called *regions*.
- When compound condition are encountered in procedural design, flow graph becomes slightly more complicated.

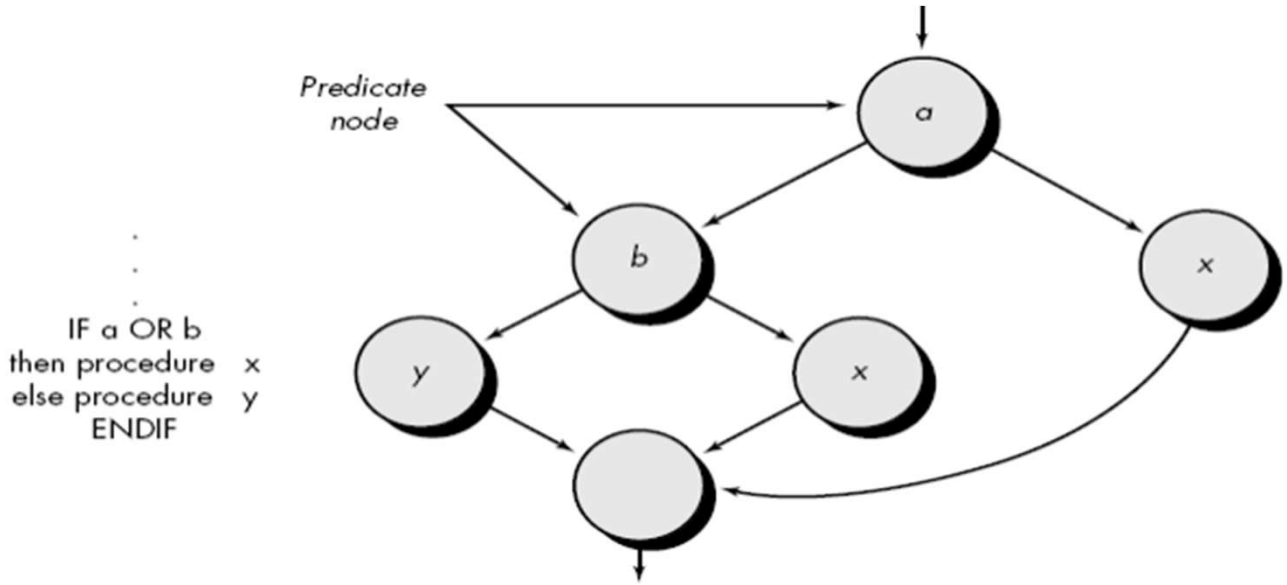
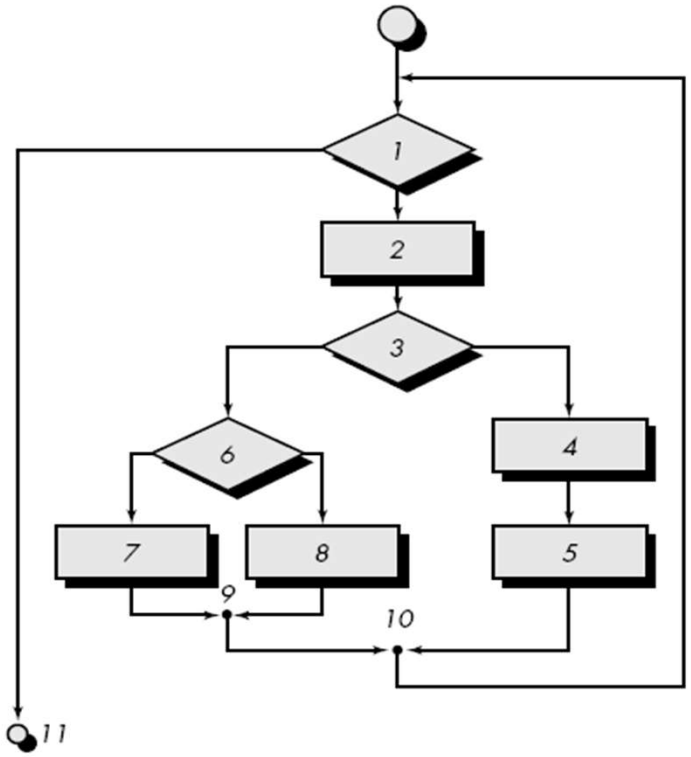
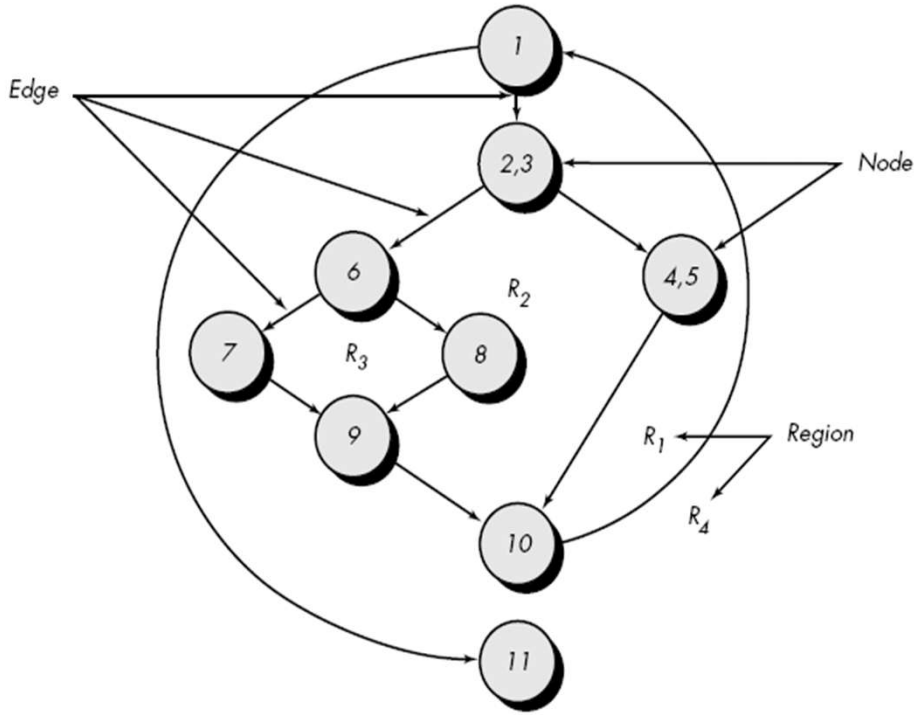


Fig. A represent program control structure and fig. B maps the flowchart into a corresponding flow graph.



(A)



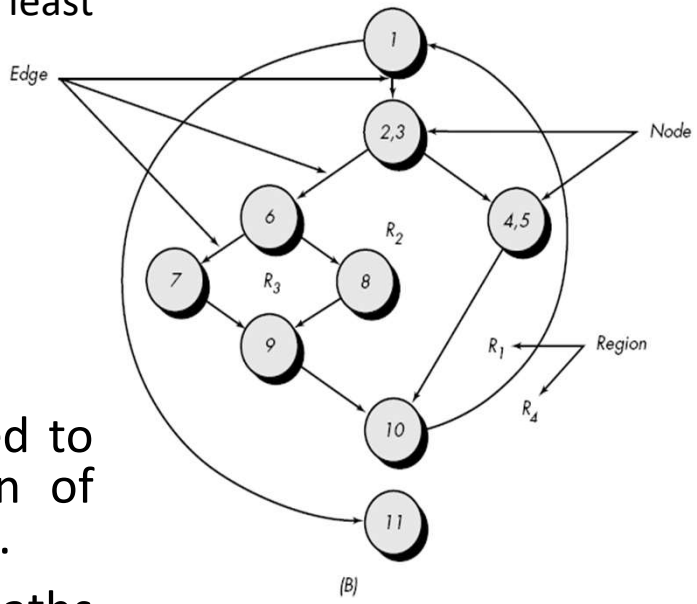
(B)

In fig. B each circle, called flow graph node, represent one or more procedural statement.

2. Independent program paths or McCabe's Cyclomatic complexity

- An *independent path* is any path through the program that introduces at least one *new set of processing statement* or *new condition*.
- For example, a set of independent paths for flow graph:
  - Path 1: 1-11
  - Path 2: 1-2-3-4-5-10-1-11
  - Path 3: 1-2-3-6-8-9-10-1-11
  - Path 4: 1-2-3-6-7-9-10-1-11

  } Basis Set
- Note that each new path introduces a new edge.
- The path 1-2-3-4-5-10-1-2-3-6-8-9-10-1-11 is not considered to an independent path because it is simply a combination of already specified paths and does not traverse any new edges.
- Test cases should be designed to force execution of these paths (basis set).
- Every statement in the program should be executed at least once and every condition will have been executed on its true and false.



How do we know how many paths to looks for ?

- *Cyclomatic complexity* is a software metrics that provides a quantitative measure of the logical complexity of a program.

"cyclomatic complexity of program = The number of independent paths in the basis set of a program"

- It defines no. of independent paths in the basis set and also provides number of test that must be conducted.
- One of three ways to compute cyclomatic complexity:
  1. The *no. of regions* corresponds to the cyclomatic complexity.
  2. Cyclomatic complexity, V(G), for a flow graph, G, is defined as

     V(G) = E - N + 2

     where E is the number of flow graph edges, N is the number of flow graph nodes.
  3. Cyclomatic complexity, V(G), for a flow graph, G, is also defined as

     V(G) = P + 1

     where P is the number of predicate nodes.

  So the value of V(G) provides us with upper bound of test cases.

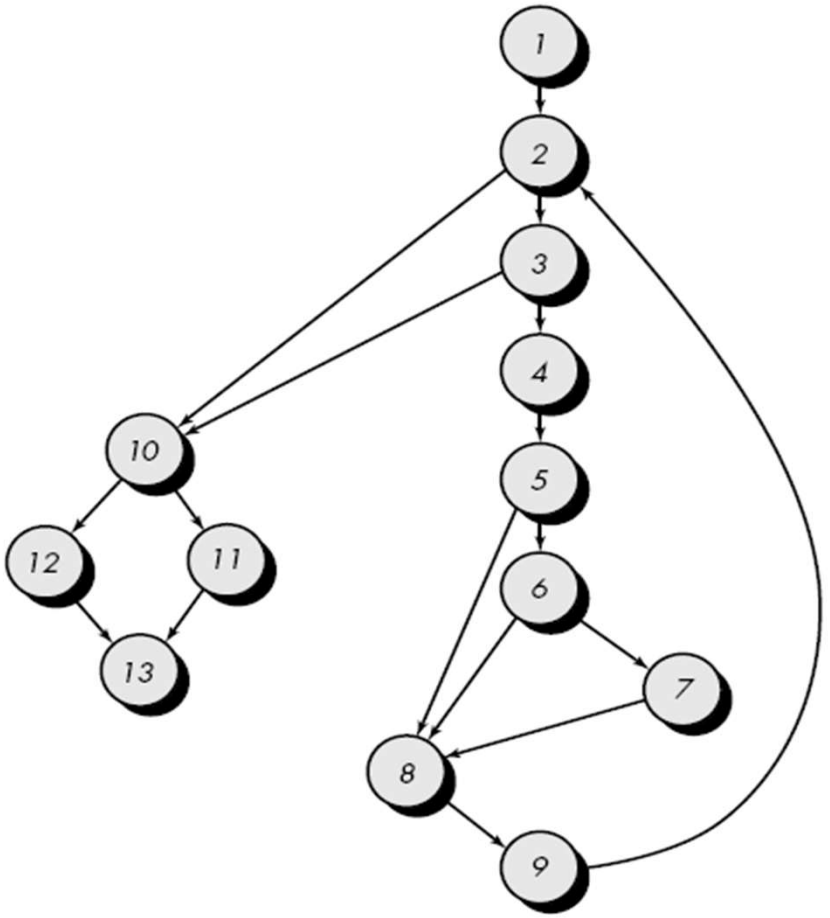
Deriving Test Cases

```
PROCEDURE average;

  * This procedure computes the average of 100 or fewer
    numbers that lie between bounding values; it also computes the
    sum and the total number valid.

  INTERFACE RETURNS average, total.input, total.valid;
  INTERFACE ACCEPTS value, minimum, maximum;

  TYPE value[1:100] IS SCALAR ARRAY;
  TYPE average, total.input, total.valid;
     minimum, maximum, sum IS SCALAR;
  TYPE i IS INTEGER;
  i = 1;                                                    (1)
  total.input = total.valid = 0;                            (2)
  sum = 0;                                                  (1)
  DO WHILE value[i] <> -999 AND total.input < 100           (3)
    (4) increment total.input by 1;
    IF value[i] > = minimum AND value[i] < = maximum        (6)
    (5)  THEN increment total.valid by 1;                   (7)
                sum = s sum + value[i]                      (7)
         ELSE skip                                          (7)
    ENDIF                                                   (8)
    increment i by 1;                                       (8)
  (9) ENDDO
    IF total.valid > 0                                      (10)
    (11) THEN average = sum / total.valid;
    (12) ELSE average = -999;
  (13) ENDIF
END average
```



Flow graph for the procedure average

2. Determine the cyclomatic complexity of the resultant flow graph.

V(*G*) can be determined without developing a flow graph by counting all conditional statements in the PDL (for the procedure *average,* compound conditions count as two) and adding 1

1. *V*(*G*) = 6 regions
2. *V*(*G*) = 17 edges - 13 nodes + 2 = 6
3. *V*(*G*) = 5 predicate nodes + 1 = 6

3. Determine a basis set of linearly independent paths

The value of *V*(*G*) provides the number of linearly independent paths through the program control structure.

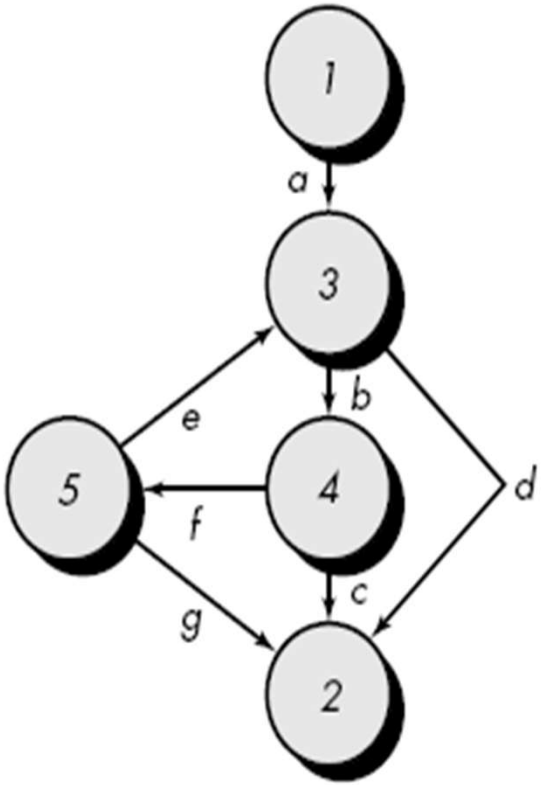
1. path 1: 1-2-10-11-13
2. path 2: 1-2-10-12-13
3. path 3: 1-2-3-10-11-13
4. path 4: 1-2-3-4-5-8-9-2-. . .
5. path 5: 1-2-3-4-5-6-8-9-2-. . .
6. path 6: 1-2-3-4-5-6-7-8-9-2-. . .

The ellipsis (. . .) following paths 4, 5, and 6 indicates that any path through the remainder of the control structure is acceptable.

4. **Prepare test cases that will force execution of each path in the basis set.**
   - Data should be chosen so that conditions at the predicate nodes are appropriately set as each path is tested.
   - Each test case is executed and compared to expected results.
   - Once all test cases have been completed, the tester can be sure that all statements in the program have been executed at least once.

4. Graph Matrices

- A *graph matrix* is a square matrix whose size (i.e., number of rows and columns) is equal to the number of nodes on the flow graph.
- Each row and column corresponds to an identified node, and matrix entries correspond to connections (an edge) between nodes.
- Each node on the flow graph is identify by numbers, while each edge is identify by letters.
- The graph matrix is nothing more than a tabular representation of a flow graph.
- By adding a *link weight* to each matrix entry, the graph matrix can become a powerful tool for evaluating program control structure during testing.
- The link weight provides additional information about control flow. In its simplest form, the link weight is 1 (a connection exists) or 0 (a connection does not exist).



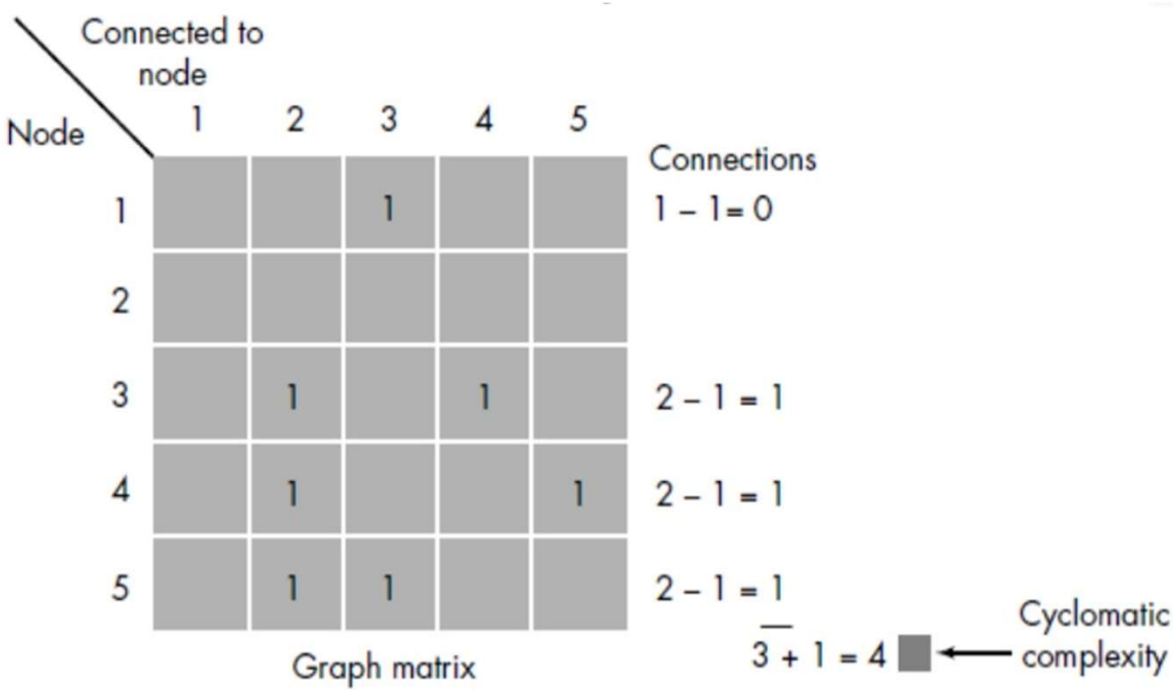
Flow graph

Node \ Connected to node	1	2	3	4	5
1			a		
2					
3		d		b	
4		c			f
5		g	e		

Graph matrix

Connection matrix

1. Count the number of 1s in each row and write it in the end of the row
1. Subtract 1 from this count for each row (Ignore the row if its count is 0)
1. Add the count of each row calculated previously
1. Add 1 to this total count
1. The final sum in Step 4 is the cyclomatic complexity of the control flow graph



Each letter has been replaced with a 1, indicating that a connection exists (this graph matrix is called a *connection matrix).*

In fig.(connection matrix) each row with two or more entries represents a predicate node.

Example

Question : Consider the given program that checks if a number is prime or not. For the following program :

1. Draw the Control Flow Graph
2. Calculate the Cyclomatic complexity using all the methods
3. List all the Independent Paths
4. Design test cases from independent paths

```
int main()
{
        int n, index;
        cout << "Enter a number: " << endl;
        cin >> n;
        index = 2;
        while (index <= n - 1) {
                if (n % index == 0) {
                        cout << "It is not a prime number" << endl;
                        break;
                }
                index++;
        }
        if (index == n)
                cout << "It is a prime number" << endl;
} // end main
```

1. Draw the Control Flow Graph

Step-1:
Start numbering the statements after declaration of the variables.

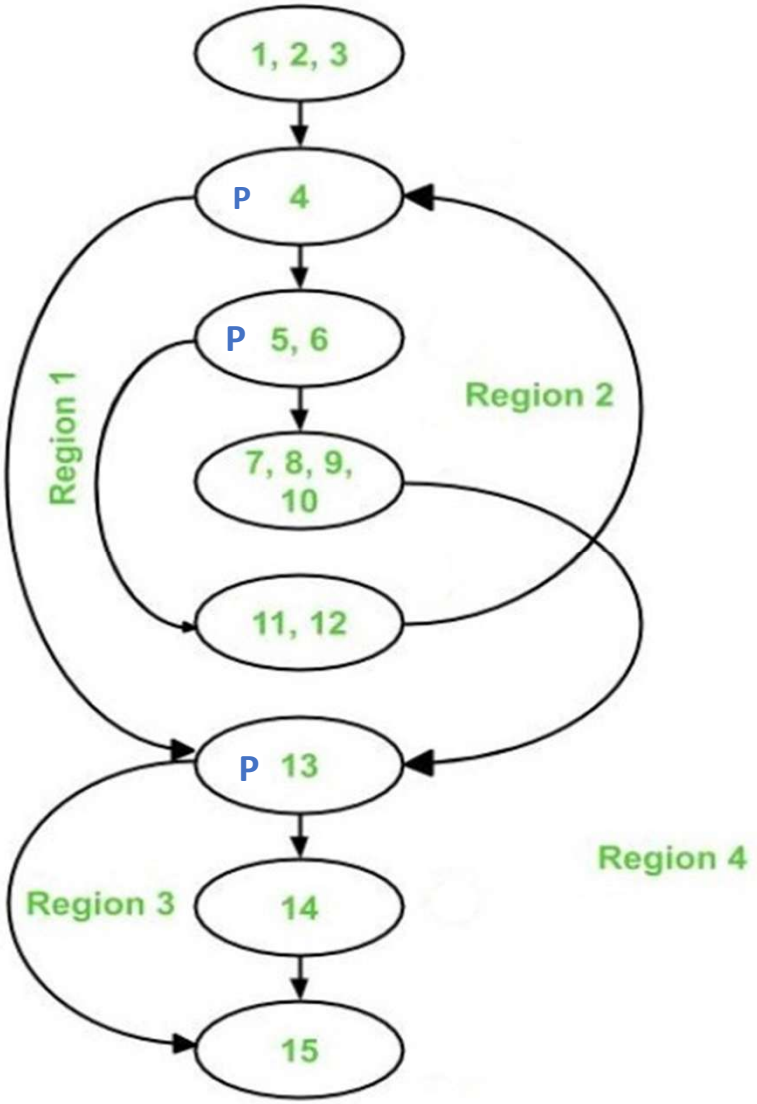
However, if a variable has been initialized and declared in the same line, then numbering should start from that line itself.

For the given program, this is how numbering will be done:

```
int main()
{
   int n, index;
1    cout << "Enter a number: " <> n;
3    index = 2;
4    while (index <= n - 1)
5    {
6        if (n % index == 0)
7        {
8            cout << "It is not a prime number" << endl;
9            break;
10       }
11       index++;
12   }
13   if (index == n)
14       cout << "It is a prime number" << endl;
15 } // end main
```

Step-2:
Put the sequential statements into one single node. For example, statements 1, 2 and 3 are all sequential statements and hence should be combined into a single node.

```
int main()
{
   int n, index;
1    cout << "Enter a number: " <> n;
3    index = 2;
4    while (index <= n - 1)
5    {
6        if (n % index == 0)
7        {
8            cout << "It is not a prime number" << endl;
9            break;
10        }
11        index++;
12    }
13    if (index == n)
14        cout << "It is a prime number" << endl;
15 } // end main
```



2. Calculate the Cyclomatic complexity :

Method-1:

V(G) = E - N + 2

E = 10 & N = 8

V(G) = 10-8+2 = 4

Method-2:

V(G) = P + 1

Here P=3

V(G) = 3+1=4

Method-3:

V(G) = Number of Regions

V(G) = 4

3. Independent Paths :

As the cyclomatic complexity V(G) for the graph has come out to be 4, therefore there are 4 independent paths.

Use alphabetical numbering on nodes for simplicity

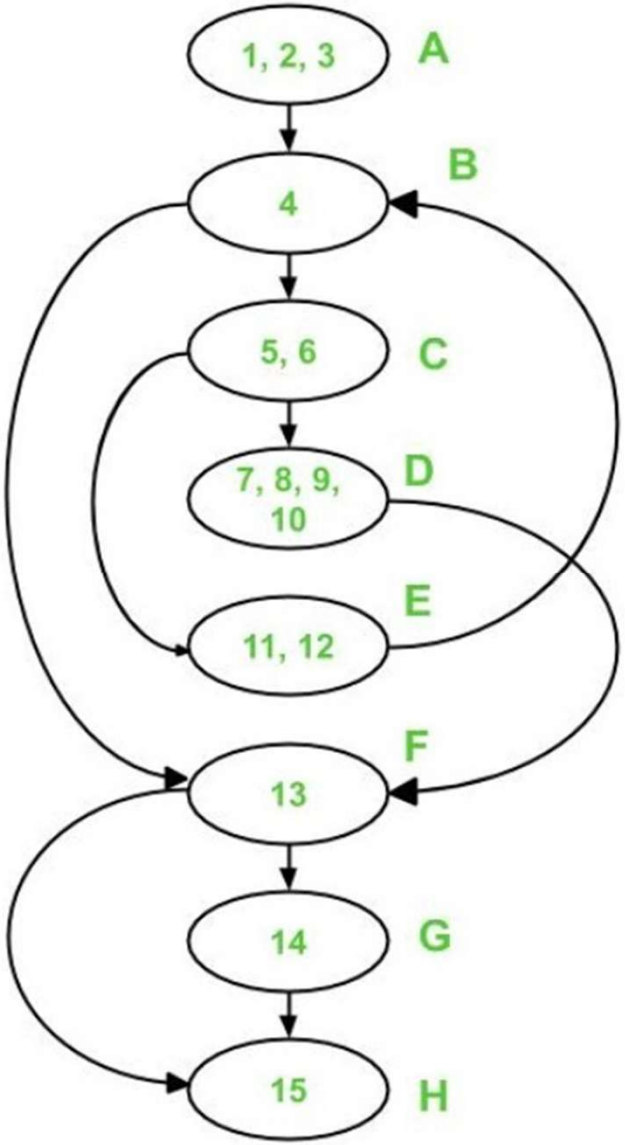
Path 1 : A - B - F - G - H

Path 2 : A - B - F - H

Path 3 : A - B - C - E - B - F - G - H

Path 4 : A - B - C - D - F - H

Each of these paths have introduced at least one new edge which has not been traversed before.



4. Test cases :

To derive test cases, we have to use the independent paths obtained previously. To design a test case, provide input to the program such that each independent path is executed.
For the given program, the following test cases will be obtained:

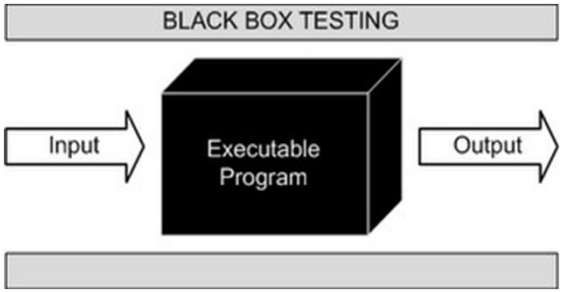
Test case ID	Input Number	Output	Independent Path covered
1	1	No output	A-B-F-H
2	2	It is a prime number	A-B-F-G-H
3	3	It is a prime number	A-B-C-E-B-F-G-H
4	4	It is not a prime number	A-B-C-D-F-H

White Box Testing

- **Advantages**
  - Forces test developer to understand carefully about implementation.
  - Reveals errors in "hidden" code.
  - Spots the Dead Code
- **Disadvantages**
  - Expensive as one has to spend both time and money to perform white box testing.
  - Possibility that few lines of code are missed accidentally.
  - In-depth knowledge about the programming language is necessary to perform white box testing.

Black box testing

- Also called *behavioral testing*, focuses on the functional requirements of the software.
- It enables the software engineer to derive sets of input conditions that will fully exercise all functional requirements for a program.
- Black-box testing is not an alternative to white-box techniques but it is complementary approach.
- Black-box testing attempts to find errors in the following categories:
  - Incorrect or missing functions,
  - Interface errors,
  - Errors in data structures or external data base access.
  - Behavior or performance errors,
  - Initialization and termination errors.



Black box testing methods

1. **Equivalence partitioning**
2. **Boundary value analysis (BVA)**
3. **Orthogonal Array Testing**

1. Equivalence Partitioning

- *Equivalence partitioning* is a black-box testing method that divides the input domain of a program into classes of data from which test cases can be derived.
- Test case design for equivalence partitioning is based on an evaluation of *equivalence classes* for an input condition.
- An *equivalence class* represents a set of valid or invalid states for input conditions.
- Typically, an input condition is either a specific numeric value, a range of values, a set of related values, or a Boolean condition.

To define equivalence classes follow the guideline

1. If an input condition specifies a *range*, one valid and two invalid equivalence classes are defined.
   - **Input range: 1 – 10 Eq classes: {1..10}, {x < 1}, {x > 10}**
2. If an input condition requires a specific *value*, one valid and two invalid equivalence classes are defined.
   - **Input value: 250 Eq classes: {250}, {x < 250}, {x > 250}**
3. If an input condition specifies a member of a *set*, one valid and one invalid equivalence class are defined.
   - **Input set: {-2.5, 7.3, 8.4} Eq classes: {-2.5, 7.3, 8.4}, {any other x}**
4. If an input condition is *Boolean*, one valid and one invalid class are defined.
   - **Input: {true condition} Eq classes: {true condition}, {false condition}**

Example 1:

A software can compute the square root of an input integer which can assume values in the range of 0 to 5000. Design 3 Equivalence Class Partitioning test cases.

There are three equivalence classes:

- The set of negative integers.
- The set of integers in the range of 0 and 5000 and
- The integers larger than 5000.

Therefore, the test cases must include representatives for each of the three equivalence classes and possible 3 test sets can be:

 Test suite 1: {-5,500,6000}

Test suite 2: {-15,900,6020}

Test suite 3: {-3,4999,5100}.

Example 2:

Design the black-box test suite for the following program. The program computes the intersection point of two straight lines and displays the result. It reads two integer pairs (m1, c1) and (m2, c2) defining the two straight lines of the form y=mx + c.

The equivalence classes are the following:

- Parallel lines (m1=m2, c1≠c2)
- Intersecting lines (m1≠m2)
- Coincident lines (m1=m2, c1=c2)

Now, selecting one representative value from each equivalence class, the test suit

{(2, 2) (2, 5)}
{(5, 5) (7, 7)}
{(10, 10) (10, 10)} are obtained.

The Commission Problem

- Rifle salespersons in the Arizona Territory sold rifle locks, stocks, and barrels made by a gunsmith in Missouri
- Lock = $45.00, stock = $30.00, barrel = $25.00
- Each salesperson had to sell at least one complete rifle per month ($100)
- The most one salesperson could sell in a month was 70 locks, 80 stocks, and 90 barrels
- Each salesperson sent a telegram to the Missouri company with the total order for each town (s)he visits
- $1 \leq$ towns visited ≤ 10, per month
- Commission: 10% on sales up to $1000, 15% on the next $800, and 20% on any sales in excess of $1800

Equivalence Class Test Cases for the Commission Problem

- Input Domain Equivalence Classes
  - Lock
    - L1 = { lock: $1 \leq lock \leq 70$ }
    - L2 = { lock: $lock < 1$ }
    - L3 = { lock: $lock > 70$ }
  - Stock
    - S1 = { stock: $1 \leq stock \leq 80$ }
    - S2 = { stock: $stock < 1$ }
    - S3 = { stock: $stock > 80$ }
  - Barrel
    - B1 = { barrel: $1 \leq barrel \leq 90$ }
    - B2 = { barrel: $barrel < 1$ }
    - B3 = { barrel: $barrel > 90$ }

Equivalence Class Test Cases for the Commission Problem

Weak Input Domain Equivalence Class Test Cases

Test Case	locks	stocks	barrels	sales	commission
WE1	35	40	45	3900	640
WE2	0	0	0	ERROR	ERROR
WE3	71	81	91	ERROR	ERROR

Strong Input Domain Equivalence Class Test Cases

Test Case	locks	stocks	barrels	sales	commission
SE1	35	40	45	3900	640
SE2	35	40	0	ERROR	ERROR
SE3	35	40	91	ERROR	ERROR
SE4	35	0	45	ERROR	ERROR
SE5	35	0	0	ERROR	ERROR
SE6	35	0	91	ERROR	ERROR
SE7	35	81	45	ERROR	ERROR
SE8	35	81	0	ERROR	ERROR
SE9	35	81	91	ERROR	ERROR
SE10	0	40	45	ERROR	ERROR
SE11	0	40	0	ERROR	ERROR
SE12	0	40	91	ERROR	ERROR
SE13	0	0	45	ERROR	ERROR
SE14	0	0	0	ERROR	ERROR
SE15	0	0	91	ERROR	ERROR
SE16	0	81	45	ERROR	ERROR
SE17	0	81	0	ERROR	ERROR
SE18	0	81	91	ERROR	ERROR
SE19	71	40	45	ERROR	ERROR
SE20	71	40	0	ERROR	ERROR
SE21	71	40	91	ERROR	ERROR
SE22	71	0	45	ERROR	ERROR
SE23	71	0	0	ERROR	ERROR
SE24	71	0	91	ERROR	ERROR
SE25	71	81	45	ERROR	ERROR
SE26	71	81	0	ERROR	ERROR
SE27	71	81	91	ERROR	ERROR

3. Boundary Value Analysis (BVA)

- Boundary value analysis is a test case design technique that complements equivalence partitioning.
- Rather than selecting any element of an equivalence class, BVA leads to the selection of test cases at the "edges" of the class.
- In other word, Rather than focusing solely on input conditions, BVA derives test cases from the output domain as well.

- If an input condition specifies a range bounded by values *a* and *b,* test cases should be designed with values *a* and *b* and just above and just below *a* and *b*.

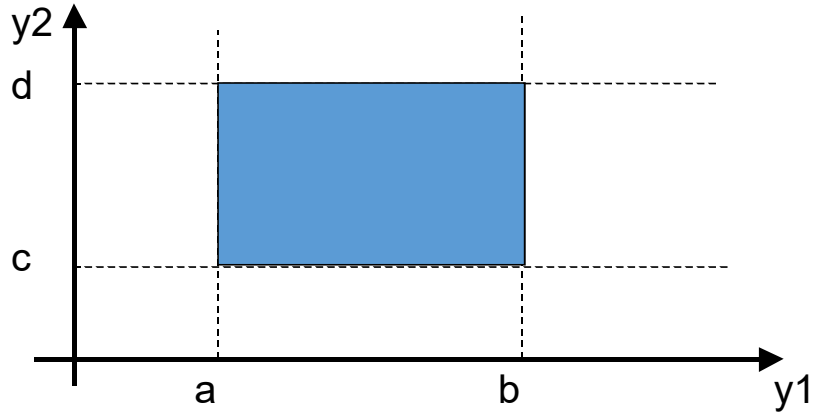
- If an input condition specifies a number of values, test cases should be developed that exercise the minimum and maximum numbers. Values just above and below minimum and maximum are also tested.

Valid Input for Program P

- consider the following function:

$$f(y_1, y_2), \text{where } a \le y_1 \le b\,, c \le y_2 \le d$$

- boundary inequalities of *n* input variables define an *n*-dimensional input space:



Value Selection in Boundary Value Analysis

- The basic idea in boundary value analysis is to select input variable values at their:
  - Minimum
  - Just above the minimum
  - A nominal value
  - Just below the maximum
  - Maximum

Boundary Value Analysis (BVA)



- Suppose system asks for "a number between **100** and **999** **inclusive**"
- The **boundaries** are **100** and **999**
- We therefore **test for values**

99	100	101

Lower boundary

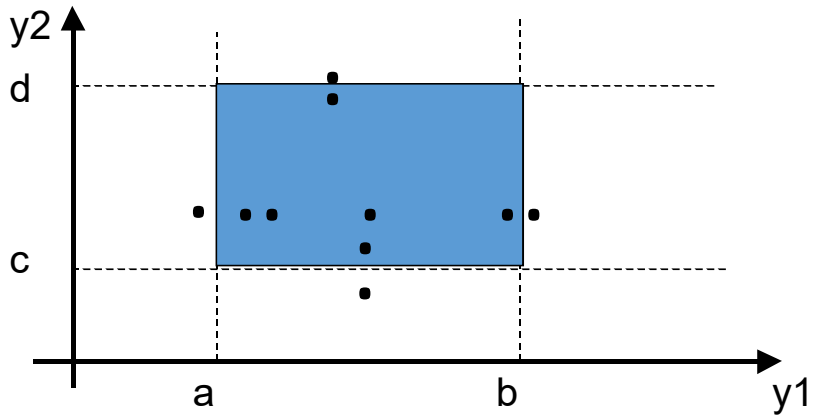
998	999	1000

Upper boundary

Generalizing Boundary Value Analysis

- The basic boundary value analysis can be generalized in two ways:
  - By the number of variables - (4n +1) test cases for n variables
  - By the kinds of ranges of variables
    - Programming language dependent
    - Bounded discrete
    - Unbounded discrete (no upper or lower bounds clearly defined)
    - Logical variables

Boundary Value Analysis for Program P



T = { <y_{1nom}, y_{2min}>, <y_{1nom}, y_{2min+}>, <y_{1nom}, y_{2nom}>, <y_{1nom}, y_{2max-}>, <y_{1nom}, y_{2max+}>, <y_{1min}, y_{2nom}>, < $_{1nin+}$, y_{2nom}>, <y_{1max-}, y_{2nom}>, <y_{1max}, y_{2nom}> }

Limitations of Boundary Value Analysis

- Boundary value analysis works well when the program to be tested is a function of several *independent* variables that represent *bounded physical quantities*.

- Boundary value analysis selects test data with no consideration of the function of the program, nor of the semantic meaning of the variables.

Independence Assumption and Efficacy of BVT

- Assumes that input variables are independent of one another,
  - i.e. the assumption that particular combinations of input variable values have no special significance
- If basic assumption is not true, then BVT may overlook important test requirements
- BVT is an instance of more general techniques such as equivalence class testing or domain testing.
- BVT tends to generate more test cases with poorer test coverage (with the independence assumption) than domain or equivalence testing.
- But, due to its simplicity, BVT test case generation can be easily automated.

Worst Case Testing

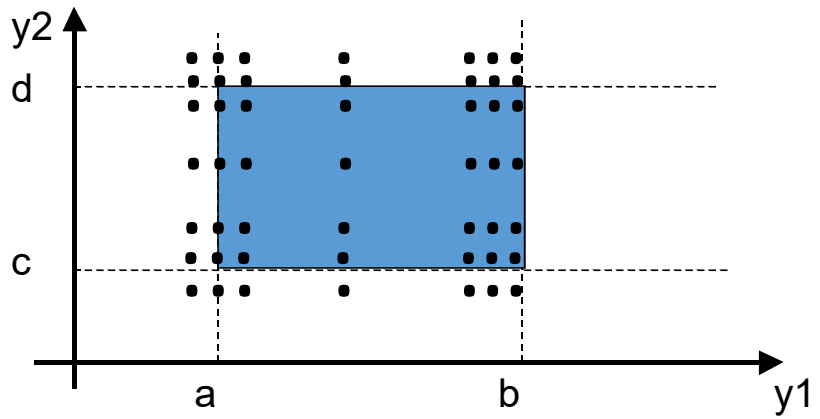
- In worst case testing we reject the single fault assumption and we are interested what happens when more than one variable has an extreme value.

- Considering that we have five different values that can be considered during boundary value analysis testing for one variable, now we take the Cartesian product of these possible values for 2, 3, … n variables.

- In this respect we can have 5^n test cases for n input variables.

- The best application of worst case testing is where physical variables have numerous interactions and failure of a program is costly.

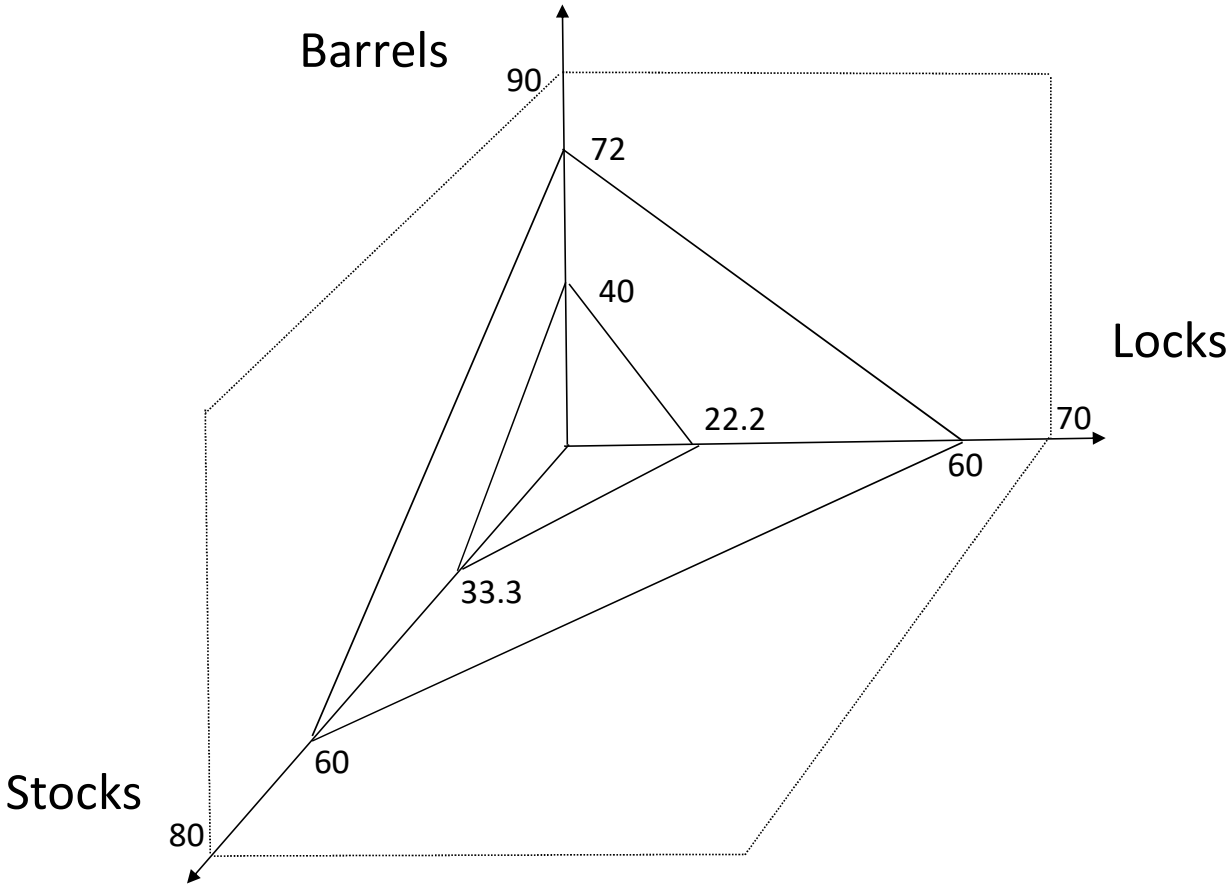
Robust Worst Case Testing for Program P



The Commission Problem

- Rifle salespersons in the Arizona Territory sold rifle locks, stocks, and barrels made by a gunsmith in Missouri
- Lock = $45.00, stock = $30.00, barrel = $25.00
- Each salesperson had to sell at least one complete rifle per month ($100)
- The most one salesperson could sell in a month was 70 locks, 80 stocks, and 90 barrels
- Each salesperson sent a telegram to the Missouri company with the total order for each town (s)he visits
- $1 \leq$ towns visited ≤ 10, per month
- Commission: 10% on sales up to $1000, 15% on the next $800, and 20% on any sales in excess of $1800

Example Test Cases Using Output Range Values



Output Boundary Value Test Cases

Case #	Locks	Stocks	Barrels	Sales	Comm.	Comments
1	1	1	1	100	10	min
2	10	10	9	975	97.5	border-
3	10	9	10	970	97	border-
4	9	10	10	955	95.5	border-
5	**10**	**10**	**10**	**1000**	**100**	**border**
6	10	10	11	1025	103.75	border+
7	10	11	10	1030	104.5	border+
8	11	10	10	1045	106.75	border+

BVA - Advantages

- The BVA is **easy to use and remember** because of the uniformity of identified tests and the automated nature of this technique.
- One can **easily control the expenses** made on the testing by controlling the number of identified test cases.
- BVA is the **best approach** in cases where the **functionality** of a software is based on numerous variables representing physical quantities.
- The technique **is best at user input troubles** in the software.
- The **procedure and guidelines are crystal clear** and easy when it comes to determining the test cases through BVA.

BVA - Disadvantages

- This technique **sometimes fails** to test **all the potential input** values. And so, the **results are unsure**.
- The **dependencies** with **BVA** **are not tested between two inputs**.
- This technique **doesn't fit** well when it comes to **Boolean Variables**.
- It **only works** well with **independent variables** that depict quantity.